

NexLink

Demo Guide

Step-by-step script for presenting the complete NexLink project — live on Azure

Item	Detail
Presenter	Maitrey Bharambe
Prepared on	25 October 2026
Live site	https://nexlink-maitrey.azurewebsites.net (Azure App Service, deployed by GitHub Actions)
Demo length	About 28 minutes + questions (11 acts; acts marked optional can be skipped)
Equipment	One Windows PC with this repository and a browser. A second computer is optional (Act 6c) — on the same network or a different one.
Device names used	Real names: this PC (hostname Maitrey) enrolls as maitrey, a second agent on it as maitrey-peer; a second computer uses its own hostname
Readiness	Live site checked on 25 Oct 2026 — see section 1

Contents

Demo at a glance ... 3
1. Readiness check (25 October 2026) ... 4
2. Two ways to present ... 5
3. The day before ... 6
  3.1 Live site (A) ... 6
  3.2 Local backup (B) ... 6
  3.3 Rehearse once ... 6
4. Starting everything (10 minutes before) ... 7
5. The demo, act by act ... 8
  Act 1 — Landing page and architecture (3 min) ... 8
  Act 2 — Sign in (1 min) ... 8
  Act 3 — Join with a code (4 min) ... 8
  Act 4 — Honest health (2 min) ... 9
  Act 5 — Protocol simulation (optional, 3 min) ... 9
  Act 6 — A real WireGuard tunnel (4 min) ... 9
  Act 7 — Real data through the tunnel (3 min) ... 10
  Act 8 — Autopsy: diagnose, approve, verify (3 min) ... 11
  Act 9 — AI Insights and device analysis (2 min) ... 11
  Act 10 — Security: revoke and audit (2 min) ... 11
  Act 11 — Quality and CI/CD (optional, 1 min) ... 11
6. Questions you may be asked ... 12
7. If something goes wrong, and resetting ... 13
  Troubleshooting during the demo ... 13
  Resetting for another run ... 13
  Documents to have ready ... 13
8. Printable checklist ... 14

Demo at a glance

The story you are telling: NexLink is a private-network platform that only reports what it can prove. Each act shows one idea, in this order:

Act	What you show	Time	Key message
1	Landing page and architecture	3 min	One app on the internet; VPN traffic goes device to device over WireGuard
2	Sign in (admin)	1 min	Sign-up is locked on the live site; roles are enforced by the server
3	Join with a code	4 min	Keys are made on the computer; nothing secret is copied; an operator approves
4	Honest health	2 min	“Enrolled” is not “connected” — no evidence means unknown
5	Protocol simulation (optional)	3 min	The packet sequence, slowed down and clearly labelled as a replay
6	A real WireGuard tunnel	4 min	Connected only after a real handshake; direct first, relay when needed
7	Real data through the tunnel	3 min	A file crosses the encrypted tunnel; same checksum at both ends
8	Autopsy: diagnose, approve, verify	3 min	Evidence, policy engine, human approval, verified result
9	AI Insights and device analysis	2 min	AI explains with receipts; it never acts on its own
10	Security: revoke and audit	2 min	A revoked device is locked out at once; everything is audited
11	Quality and CI/CD (optional)	1 min	128 automated tests; every push to main is tested and deployed



The architecture you explain in Act 1 (also in the Project Report).

1. Readiness check (25 October 2026)

Everything below was checked on the live site or this PC. Use it to answer “does this really work?”

Component	Check	Result
Live site	https://nexlink-maitrey.azurewebsites.net/healthz	{"ok":true}; pages 0.15–1.2 s when warm
CI/CD	GitHub Actions on push to main	Passed: tests → build → deploy → health check
Admin account	maitrey.bharambe24@gmail.com	admin, full permissions
Public sign-up	ALLOW_SIGNUP=false on Azure	Locked (SIGNUP_DISABLED)
Enrollment through Azure	join → code → approve on the live site	maitrey, maitrey-peer enrolled
Live tunnel, direct	maitrey-bharambe-net	Operational, 1/1 verified, 1.2–1.4 ms
Data, direct path	50 MB file through the tunnel	4.3 s, ~97 Mbit/s, checksum identical
Data, relay via Azure	10 MB file through the relay	36.5 s, ~2.3 Mbit/s, checksum identical
AI briefing	Groq openai/gpt-oss-120b, live	Correct, 7 cited findings
AI device analysis	maitrey	Sound; no actions proposed (see note)
Automated tests	web 85 · Go 28 · Python 15	128 / 128 passed
Agent binary	nexlink-agent.exe version	0.2.0

Be careful

Two physical PCs on **different** networks have not been tested yet; the demo is the first run. Expect the relay there (slower, still encrypted).

The agents run only while their PowerShell windows are open. Closing a window disconnects that device; running the same command again reconnects it.

The AI is grounded but not perfect: in one analysis it called a 10–23 ms control-plane round trip “sub-millisecond”, and it does not yet see short relay episodes. Say so if asked — it is honest.

2. Two ways to present

	A · Live site (recommended)	B · Local backup (no internet)
Address	https://nexlink-maitrey.azurewebsites.net	http://localhost:3000
Start-up	Nothing to start — just sign in	cd web; npm run dev
Account	Sign in as admin (sign-up is locked)	Fresh database → sign up → first account is admin
Agents use	--server https://nexlink-maitrey.azurewebsites.net	--server http://127.0.0.1:3000
Second PC	Any network (internet only)	Same Wi-Fi only

3. The day before

3.1 Live site (A)

- Open https://nexlink-maitrey.azurewebsites.net and sign in. Check the top-right shows your name with **admin**.
- Decide which network to use (e.g. **maitrey-bharambe-net**). To start the demo with an empty network, revoke the old devices on **devices**.
- If the site was idle, the first page load can take up to 30 s; open https://nexlink-maitrey.azurewebsites.net/healthz once to wake it.
- Delete old agent folders on this PC you no longer need (networking-agent\bin\live\…); new ones are created by the join command.

3.2 Local backup (B)

```
git pull
.\scripts\start-all.ps1 -Install
# web\.env: MONGODB_URI=mongodb://127.0.0.1:27017/nexlink_demo   (fresh database)
# web\.env: GROQ_API_KEY=gsk_…   (optional, enables AI)
```

3.3 Rehearse once

- Run the whole guide once, then revoke the rehearsal devices.
- Browser zoom 80–90% so whole pages fit on the projector.
- Keep the Project Report (PDF) open in a tab for questions.

4. Starting everything (10 minutes before)

Browser

Open https://nexlink-maitrey.azurewebsites.net/healthz (should show {"ok":true}), then https://nexlink-maitrey.azurewebsites.net and sign in.

PowerShell — agents

```
cd D:\projects\cn\networking-agent\bin
.\nexlink-agent.exe version     # prints 0.2.0
```

Optional — a file to share for Act 7

```
mkdir D:\nexlink-share; cd D:\nexlink-share
# put any file here, e.g. a 50 MB video or zip
python -m http.server 8099 --bind 127.0.0.1
```

5. The demo, act by act

Act 1 — Landing page and architecture (3 min)

1. Open https://nexlink-maitrey.azurewebsites.net — the landing page: “your devices. your network. no limits.”
2. Point at the tunnel readout and the strip at the bottom: real measured values, the 180-second health rule, WireGuard’s ChaCha20-Poly1305.
3. Show the architecture figure from this guide.



The landing page.

> **What to say**
>
> “NexLink lets you create private networks between your own computers, from anywhere, see their real connectivity, diagnose problems and run approved actions.”
>
> “It is one app on the internet — console, API, live updates and a relay — deployed on Azure. Every push to GitHub is tested and deployed automatically.”
>
> “The server only coordinates. VPN traffic goes directly between devices over WireGuard; when a firewall blocks that, it goes through a relay that only ever sees encrypted packets.”

Act 2 — Sign in (1 min)

1. Click **Sign in** → sign in as admin. Point out **admin** under your name and the **audit log** in the sidebar.
2. Mention that self-service sign-up is switched off on the live site, so nobody else can create an account.

> **What to say**
>
> “The first account became the administrator; after that I locked sign-up. Roles are enforced by the server, not just hidden in the interface.”

Act 3 — Join with a code (4 min)

In PowerShell (on another computer this is the one line from **add a device**):

```
.\nexlink-agent.exe join --server https://nexlink-maitrey.azurewebsites.net --state-dir
.\stage\maitrey
```

1. The window prints a short code (e.g. **KXRT-4P2M**), the computer name “Maitrey” and a key fingerprint.
2. In the browser: **add a device** → enter the code → check the computer name → name **maitrey** → choose the network → **approve & enroll**.
3. The agent prints “approved: enrolled as maitrey … address 10.x.x.2”.
4. Repeat with a second folder and name it **maitrey-peer**:

```
.\nexlink-agent.exe join --server https://nexlink-maitrey.azurewebsites.net --state-dir
.\stage\maitrey-peer
```

What to say

“The agent made its own keys on the PC; the private keys never leave it.”

“The line is the same for every computer and contains nothing secret. The code proves which computer is asking, and I decide whether it gets in.”

Act 4 — Honest health (2 min)

1. **devices**: maitrey and maitrey-peer show enrolled, with no heartbeat yet.
2. **topology**: a dotted “configured” line, not a lime one.

What to say

“Enrolled, but nothing has proven they can talk — so NexLink says unknown, not healthy.”

Act 5 — Protocol simulation (optional, 3 min)

1. **simulation**, scenario normal tunnel; narrate enrollment → peer config → handshake → encrypted traffic → telemetry.
2. Switch to **control plane down**: probes keep flowing through the tunnel.

Act 6 — A real WireGuard tunnel (4 min)

```
.\nexlink-agent.exe run --userspace --server https://nexlink-maitrey.azurewebsites.net --state-dir
.\stage\maitrey --listen-port 51850
.\nexlink-agent.exe run --userspace --server https://nexlink-maitrey.azurewebsites.net --state-dir
.\stage\maitrey-peer --listen-port 51851 --share http:8099
```

Be careful

Start the second window a few seconds after the first, so the agents find each other directly. If both start together they may use the relay — still encrypted; restarting maitrey-peer gives the direct path.

Windows may ask about network access for nexlink-agent.exe — click Allow.

1. Each window prints “tunnel peers updated … (direct)” and “tunnel probe … rtt_ms≈1.3 handshake=1s ago”.
2. **networks** → **your network**: within a minute, **OPERATIONAL — 1/1 peer pairs verified by fresh handshake**; both devices healthy.
3. **topology**: the line turns solid lime, labelled **direct**, with packets moving along it.

What the audience sees

For the first minute the console may still say “last handshake: never” — the first report was sent before the handshake finished. It changes at the next report.

What to say

“This is the real WireGuard protocol running inside the agent. The control plane is in Azure; the tunnel runs directly between the devices.”

Optional 6b — the relay

Restart maitrey-peer with `--force-relay`. Topology labels the link **relay**; the probe time rises to about 25–50 ms because each packet now goes to Azure and back.

Optional 6c — a second computer, same or different network

Second PC is…	Path	What to expect
On the same Wi-Fi / LAN	Direct UDP	Fastest. Windows asks to allow network access — Allow.
On a different network (home, college, hotspot)	Direct if the routers allow UDP, otherwise the relay	Always connects (the relay uses HTTPS). Over the relay expect ~2–3 Mbit/s and 25–50 ms.

1. On the second computer, open PowerShell (no admin) and run:

```
irm 'https://nexlink-maitrey.azurewebsites.net/join.ps1' | iex
```

1. It shows a code with that computer’s own name. Approve it on **add a device**.
2. Topology shows the new link as **direct** or **relay** within a minute.

What to say

“The second computer needed only internet access and one pasted line — no admin rights, no VPN app, no router settings.”

Act 7 — Real data through the tunnel (3 min)

maitrey-peer shares the folder served on 127.0.0.1:8099 (`--share http:8099` in Act 6). From maitrey, open a session to it:

```
.\nexlink-agent.exe session --list --state-dir .\stage\maitrey
.\nexlink-agent.exe session maitrey-peer http --state-dir .\stage\maitrey
```

1. `session --list` shows **maitrey-peer · online/direct · http:8099**.
2. The session command opens the shared folder in the browser through a local port; click the file to download it.
3. Point at the download speed. Optional: compare checksums with `Get-FileHash <file>` on both copies — they are identical.

What the audience sees

In rehearsal: 50 MB in 4.3 s (~97 Mbit/s) over the direct path; 10 MB in 36.5 s (~2.3 Mbit/s) through the Azure relay — both with identical checksums.

What to say

“This file travelled inside the WireGuard tunnel, encrypted end to end. The server never saw it — even on the relay it only forwards ciphertext.”

Act 8 — Autopsy: diagnose, approve, verify (3 min)

1. **autopsy** → **test event** → device maitrey, category peer handshake stale, peer maitrey-peer → create.
2. Open the [TEST] incident: investigation states, path view and checks; **run diagnostics** (and **ask ai**).
3. On run_connectivity_test click **approve & run** → **verified outcome: successful — 3/3 measurements valid**.

What to say

“Diagnosis uses read-only tools. Every proposed action passes a policy engine and needs a human to approve it; the result is verified from fresh measurements.”

Act 9 — AI Insights and device analysis (2 min)

1. **ai insights** → **regenerate** for your network: summary, findings F-01… each with its evidence reference, risks with plausible / untested tags, uncertainties.
2. **devices** → **maitrey** → **analyze with AI**: a grounded health analysis that proposes no actions.

What to say

“Every finding cites a recorded fact; a citation that does not exist is rejected. The AI explains, it never acts — and it never receives private keys, passwords or session tokens.”

“It is not perfect: it sees summaries, so a short relay episode can slip past it. That is the next improvement.”

Act 10 — Security: revoke and audit (2 min)

1. **devices** → **maitrey-peer** → **revoke** → reason “lost laptop”.
2. Its window immediately reports DEVICE_REVOKED; the topology link disappears.
3. **audit log**: enrollment, approvals, verification, revocation and the denied agent request.
4. Close both agent windows.

What to say

“Revocation takes effect on the device’s very next request, and its relay connection is cut. Every security-sensitive event is written to an append-only audit log.”

Act 11 — Quality and CI/CD (optional, 1 min)

1. Open github.com/Maitrey-Bharambe/nexlinkk → **Actions**: green runs of “CI/CD” — web tests, agent tests, build and deploy to Azure, health check.
2. Explain that GitHub signs in to Azure without a stored password (OpenID Connect).

What to say

“There are 128 automated tests across Node.js, Go and Python. Every push to main is tested, built and deployed automatically, then checked live.”

6. Questions you may be asked

Question	Answer
Do both PCs have to be on the same network?	No. Each PC only needs internet access to reach the portal. On the same LAN they connect directly; on different networks they connect directly if the routers allow UDP, otherwise through the encrypted relay.
Why doesn’t VPN traffic go through your server?	It would be a bottleneck and a privacy risk. The server distributes keys, addresses and policy; WireGuard connects devices directly. Only when that is impossible does the relay forward packets — still encrypted end to end.
Why is the relay so much slower?	Each packet travels to Azure and back inside a WebSocket over the internet; TCP inside that tunnel slows down on a 25–50 ms round trip. It is the fallback — direct is always tried first.
Once on the relay, does it switch back to direct?	Not while the relay link keeps working — a known gap. Restarting an agent after its peer has published its address gives a direct path.
Is this real WireGuard or a simulation?	Real: the agent embeds wireguard-go, the official userspace implementation. The simulation page is a separate, labelled replay.
Why is joining safe if the command has no secret?	The computer signs its code request with its own key, and nothing happens until an operator approves the code after checking the computer name and key fingerprint.
Where is it hosted and how is it deployed?	Azure App Service (Node 22) with MongoDB Atlas. GitHub Actions tests every push and deploys main, signing in to Azure with OpenID Connect — no stored password.
What does the AI actually do?	It reads evidence collected by read-only tools and writes briefings and diagnoses. Output must match a schema and cite real evidence; actions must be allowlisted, pass the policy engine and be approved by a human.
Is the AI always right?	No. Citations are checked, numbers are not yet; in testing it once misdescribed a control-plane round trip. It is advisory, and networking never depends on it.
How did you test it?	128 automated tests, real tunnel runs (direct and relay), file transfers with matching checksums, and a live deployment checked end to end on 25 October.

7. If something goes wrong, and resetting

Troubleshooting during the demo

What you see	What to do
The site is slow to open	The first request after a restart takes up to 30 s. Open https://nexlink-maitrey.azurewebsites.net/healthz and wait.
The site shows an error page	Azure portal → nexlink-maitrey → Log stream; or present from the local backup (section 2).
The join code expired	Codes last 15 minutes — run the join command again.
“already enrolled as …”	That --state-dir is already used — pick a new folder.
Console says “last handshake: never”	Wait for the next report (up to a minute) and refresh.
Link shows relay although both agents are on this PC	They started at the same moment. Restart maitrey-peer — or use it to explain the relay.
session says “shares nothing”	Restart maitrey-peer with --share http:8099 and keep the python file server running.
Second computer stays unreachable	Check it has internet and that its agent window is still open; on the same Wi-Fi allow the Windows network prompt.
Approved test shows “timed out”	The agents were not running when you approved — start them and approve a new test.

Resetting for another run

1. Close every agent window.
2. Revoke the demo devices on **devices**.
3. Delete networking-agent\bin\stage (it contains the demo agents’ private keys; git ignores it anyway).

Documents to have ready

Document	Use it for
docs/NexLink-Project-Report.pdf	Full report: architecture, status, test results
docs/deploy.md	How the Azure deployment and CI/CD are set up
docs/run-commands.md	Starting locally, adding computers, remote sessions
docs/architecture.md, network-protocols.md, threat-model.md	Deeper architecture, protocol and security questions

8. Printable checklist

- ☐ https://nexlink-maitrey.azurewebsites.net/healthz shows ok; signed in as admin
- ☐ Old devices revoked if you want an empty network
- ☐ PowerShell open in networking-agent\bin; agent version 0.2.0
- ☐ Optional: file in D:\nexlink-share and python file server on 8099
- ☐ Browser zoom 80–90%; Project Report open in a tab
- ☐ Act 3: maitrey and maitrey-peer approved
- ☐ Act 6: both agents running; network operational; topology direct
- ☐ Act 7: file downloaded through the session
- ☐ Act 8: approved test verified successful
- ☐ Act 10: revoke shown, audit log shown, agent windows closed
- ☐ Backup ready: local start (cd web; npm run dev) if the internet fails